ML-KEM Legacy Modernization

Cryptography and Mathematics

Table of Contents

Introduction – The need for ML-KEM....2
Changing the mathematical perspective....2
Some ML-KEM tweaks in real implementations....4
Impacts on our project....5
Performance Impact....5
Cryptographic Impact....6
References....7

Abstract

This document aim to examine the project from the cryptography and mathematical standpoints. We first start by introducing the mathematic behind ML-KEM at a high level, without diving too much into the mathematical details and then we analyze the consequences of integrating ML-KEM into the system.

Since fully describing the mathematics behind the scheme would require an entire book, we have intentionally abstracted away complex topics such as polynomial rings, the NTT algorithm, and the binary encoding/decoding of the messages. This allows us to maintain the document's core focus.

In order to keep our discussion as simple as possible we report only the notions that allow us to explain why mechanism like DH are weak in a future post-quantum era and why schemes like ML-KEM will very likely replace them.

Introduction – The need for ML-KEM

Given the possible beginning soon or later of a post-quantum era, existing cryptographic algorithms like the Diffie-Hellman key exchange implemented in our baseline system are at risk. With quantum computing, new quantum algorithms, like Grover's algorithm and Shor's algorithm arise.

In particular the Diffie-Hellman key exchange that our baseline system uses, relies on the Discrete Logarithm Problem, which consists of finding a value α, given a known value A, such that:

$$A = g^{\alpha} \bmod p$$

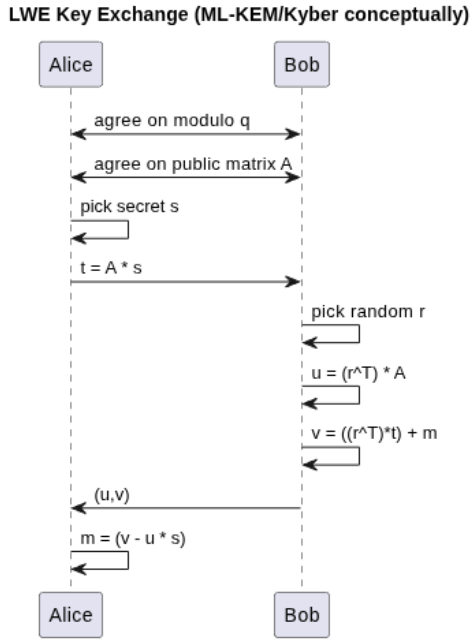
with g a given number and p a very large prime number.

However Shor's algorithm has demonstrated that it can reduce these problems to a period-finding problem. Thus, It transforms the discrete logarithm problem into an instance of a period-finding problem in a specially constructed function.

It is then crucial to replace algorithms like this with post-quantum resistant ones like ML-KEM.

Changing the mathematical perspective

We already stated how quantum computers are able to violate classical key-exchange schemes that rely on big prime numbers to be secure, like our DH scheme. ML-KEM, instead, follows another philosophy. First of all we replace the large numbers with matrices, so now the two parties do not agree on a common base g but on a common matrix A. Each party then generates a secret that is a vector. At a very high and informal level the key exchange works as represented in the picture:



The problem is that in this way it is trivial to retrieve the secret key s from just intercepting t and at the same time it is trivial to obtain the original plaintext m by just intercepting u.

The solution adopted in ML-KEM consists of introducing small errors in the computation of t, u and v, in this way:

- $t = A * s + e_1$
- $u = r^T * A + e_2$
- $v = r^T * t + e_3 + m$

Now an attacker that intercept any of these messages cannot retrieve either the secret vectors s or r nor the plaintext m.

All the error vectors must remain secret.

The correctness of the decryption is made possible thanks to the way the plaintext m is encoded and decoded.

The strength of the system relies on the following hard problem, known as the LWE (Learn with error) problem:

Given the vector *t*, an attacker should not be able to say from which of the following distributions:

- *(A, t = A * s + e)* where *s* and *e* are vectors of dimension *n* and their elements picked from [-β, β+] for properly chosen β.
- (*A,t)* with *t* vector of dimension *n* and elements picked from [0, q-1].

In more than 50% of the cases.

In other word, given a vector *t*, the attacker must not be able to say if we generated it following the first rule or if it is a randomly picked vector from all the possible ones.

If an attacker is able to guess the distribution by just seeing *t* then they will be able to just find *s*, even on a normal computer.

Now, let us try to look at the same concept but from a different prospective: computing *A*s =t* yields a vector, we can plot this vector and it will "land" on a point. This is the rough idea to switch from algebra to lattices.

A lattice is just an infinite grid (multidimensional when it comes to PQC applications) of points that always follow a clear pattern. Then *t* is just a point on this grid.

Once we add an error *e* to *t* we are just pushing the point slightly off the grid. In this context the goal of the attacker is to find the original point, that in other terms is the closest point of the grid to *t*, this problem is known as the Closest Vector Problem (CVP). Be aware that LWE and CVP are not different problems, they are the same but under different perspectives. In an informal way we can say that CVP is the geometric perspective while LWE is an algebraic wrapper for CVP.

In a 2D lattice solving this problem is easy but when the dimension is big enough this problem becomes hard, indeed is NP-hard.

Finally we need to discuss the nature of the error vectors e_i that are introduced by the scheme. It is important to note that they are not chosen at random since in this case they would overwhelm the equations making the decryption impossible even for the legitimate recipient. Then ML-KEM draws these errors from a specific distribution that statistically behaves as a bell curve (something like a Gaussian Distribution), the result is that the chosen errors are generally small, larger errors are possible but this event is extremely rare. If these errors are small the recipient can round them away (the mechanism to do this relies on the way the plaintext is encoded and decoded in a sequence of bits) when retrieving the plaintext, but they remain mathematically unpredictable enough that an attacker cannot factor them out.

In theory there is a very small probability that the legitimate recipient will not be able to retrieve the correct plaintext, resulting in a decryption failure. According to the official NIST FIPS 203 specification, this rate is something like 2^{-164} for the version of ML-KEM we have implemented (ML-KEM768). So, in practice this should not be a problem but mechanisms that take into account this inconvenience exist and consist of retrying the handshake in case of failure.

Some ML-KEM tweaks in real implementations

The naive approach that we described has a little performance problem: each encryption requires $O(n^2)$ multiplication, where n is the dimension of the matrix, since making this scheme secure by relying only on numbers would require matrices so big that computing a single encryption would take too much time to be feasible for actual usage, real implementations of this mechanism use polynomial rings.

This means that the entries of the matrix are no longer numbers modulo q but polynomials, ML-KEM768 uses polynomials modulo $X^{256}+1$.

In this way the plaintext is encoded as a polynomial, each coefficient encodes a bit, since the modulo now is the polynomial $X^{256}+1$ we can encode ciphertext blocks of size 256 bits.

This solves the performance issue, since now we can rely on a smaller matrix that is actually a 3x3 full of polynomials.

Moreover the modulo q will be the modulo for the coefficients. Usually $q = 3329$. This particular value allows efficient multiplications via the NTT algorithm, actually making the key exchange even faster than the one based on elliptic curves.

Impacts on our project

The mathematical framework on which ML-KEM is based has several consequences on both the performance-side and the cryptography-side of the system.

Performance Impact

First of all let us recall the metrics stated in the *Critical Evaluation and Maintenance* document regarding the implementation of ML-KEM. The situation before the introduction of ML-KEM was this:

Metric	Value
Handshake Latency (ms)	73.3503
End-to-End Latency (ms)	0.1529
Gateway Public Key Size (Bytes)	256
Cloud Public Key Size (Bytes)	256
Cipher text Size (Bytes)	48
Re-encrypted Cipher text Size (Bytes)	48

Let us now compare these metrics with the one recorded after the implementation and execution of ML-KEM, that are the following:

Metric	Value
Handshake Latency (ms)	75.8205
End-to-End Latency (ms)	8.1892
Gateway DH Public Key Size (Bytes)	256
Gateway ML-KEM EK Size (Bytes)	1184
Gateway Total Payload Size (Bytes)	1440
Cloud DH PK Size (Bytes)	256
Cloud ML-KEM CT Size (Bytes)	1088
Cloud Total Payload Size (Bytes)	1344
Ciphertext Size (Bytes)	32
Re-Encrypted Ciphertext Size (Bytes)	43

We can notice that in a real scenario there would be a significant bandwidth overhead on our communication. As demonstrated by our metrics, the classical Diffie-Hellman public key requires only 256 Bytes of payload while the ML-KEM encapsulation key (EK) requires 1184 Bytes and the ciphertext (CT) requires 1088 bytes.

Moreover because we implemented a Hybrid Scheme that combines both DH and ML-KEM, our Edge gateway's total payload, consisting of both the ML-KEM EK and DH public key jumped to 1440 and the Cloud's payload jumped to 1344 Bytes. To put the number in perspective this is about a 5.6x increase with respect to the original baseline system.

Regarding computational efficiency, despite the massive increase in payload size, the mathematical properties of ML-KEM is able to ensure that the system remain operationally efficient. Since ML-KEM relies on the NTT algorithm (Number Theoretical Transform) for polynomial multiplication, the computational complexity remains low. Our empirical test shows that the hybrid handshake latency average is about 75.8 ms versus the 73.3 ms of the baseline system but the end-to-end latency spiked up to 8.1 ms from being practically zero in the baseline system. Despite this, the system remains usable, even for limited edge devices.

Cryptographic Impact

There is a reason why ML-KEM has been integrated into the system rather than entirely substituting the DH key exchange.

This hybrid scheme is a recognized cryptographic best practice in the current post-quantum transition period. Despite appearing a necessary complexity carrying out two computations that can be substituted by only one the system in this way is assured against possible vulnerabilities of ML-KEM. Let us consider the following scenarios to understand this approach:

- In the future quantum computers will be able to solve the Discrete Logarithm problem and then break the DH scheme. If ML-KEM is a strong and reliable algorithm the confidentiality of our communication will still be protected since there is no quantum algorithm that can solve LWE in polynomial time.
- If before the advent of quantum computers someone finds out that ML-KEM is broken, i.e. an algorithm that solve the LWE problem is found, then an attacker would still need to find a way to break the DH exchange in order to violate the confidentiality of our communication.

In other words the catastrophe for our system (an attacker is able to derive both shared secrets) happens only if both the two previous conditions are met: someone discovers a flaw in ML-KEM and that someone has access to a quantum computer able to break the DH key exchange.

The final AES key is derived by concatenating the classical DH secret and the secret provided by ML-KEM. The two concatenated secrets are then fed to a Key Derivation Function to obtain a valid AES key.

While the mathematical structure of ML-KEM inherently introduces bandwidth overhead due to larger key sizes and the theoretical possibility of decryption failures, optimizations like NTT algorithm ensure that the computational burden remains highly viable. Finally, by implementing a hybrid architecture, we ensure that our system is robustly protected against both the classical attackers of today and the quantum threats of tomorrow.

References

https://blog.cloudflare.com/lattice-crypto-primer/

https://csrc.nist.gov/pubs/fips/203/final